RetroC: A Compiler for Retroactively Editable Dataflow Pipelines with Verified Record-Level Recomputation

Swaraj Joshi
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
Vellore-632014, Tamil Nadu, India
swaraj.joshi2024@vitstudent.ac.in

Sudeep Kumar Sahu
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
Vellore-632014, Tamil Nadu, India
sudeep.kumarsahu2024@vitstudent.ac.in

Johan Viji
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
Vellore-632014, Tamil Nadu, India
johan.vijiskariah2024@vitstudent.ac.in

Krishnaraj N
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
Vellore-632014, Tamil Nadu, India
krishnaraj.n@vit.ac.in

***Abstract*—Data pipelines are routinely corrected after they have run: a faulty input record is fixed, a record is added or withdrawn, a missing cleaning stage is inserted, or an obsolete stage is removed. Common practice either reruns the whole pipeline or reruns every stage downstream of the change, while incremental dataflow engines that avoid this fix the shape of the computation when it is built. This paper presents RetroC, a compiler and runtime for a pure, statically typed dataflow language in which retroactive edits—modifying, inserting or deleting an input record, and inserting or deleting a stage—are first-class, type-checked, atomic operations on a versioned dataflow graph called the Retroactive IR. After an edit, RetroC propagates a record-level work set: each stage visits only the records whose inputs changed, reuses per-record memo entries keyed by input and function hashes, maintains integer sums and hash joins incrementally, updates an order-independent collection hash in constant time per changed record, and stops at the first stage whose output is unchanged. A lightweight cost model chooses, per stage, between patching and recomputing. Every result can be checked against a full rerun of the textually rewritten program. On four synthetic workloads measured with a pinned, interleaved protocol (three warm-up runs, median of eleven), a single-record edit runs 525–53,400× faster than a full rerun from 10 k to 1 M records and 10,617× faster on a 12-stage pipeline, and batches of sequential edits run 6.23–1,065× faster. All 35 workload–edit scenarios are faster than a full rerun (1.08–53,400×); they beat stage-level invalidation in 33 of 34 scenarios and are on par (within 2%) in the remaining one, with gains up to 101,838×. Correctness was confirmed on 10,000 randomized programs and edit scripts and by the verifier on every benchmark scenario.**

Keywords—incremental computation, dataflow programming, compilers, intermediate representation, memoization, change propagation, differential testing

I. Introduction

Dataflow pipelines—chains of *map*, *filter*, *reduce* and *join* stages over collections of records—are the backbone of extract-transform-load jobs, feature engineering and scientific data analysis. They are rarely written once and left alone. A sensor log turns out to contain a corrupt reading; an analyst realises that values must be calibrated *before* they are aggregated; a debugging stage becomes obsolete. Each of these is a *retroactive* edit: it changes the history of a computation that has already run, and the user wants the outputs that the edited pipeline would have produced.

The usual way to obtain those outputs is to run the pipeline again. Batch engines recompute every stage for every record [1], and workflow orchestrators and build tools track dependencies between whole stages or files, so a single changed record invalidates every downstream stage in full [2]; [3]. Incremental computation research shows that far less work is needed when only a small part of the input changes [4]; [5], and incremental dataflow engines propagate *differences* through a dataflow graph with impressive efficiency [6]; [7]. These engines, however, fix the topology of the computation when it is built: inserting or deleting a stage is outside their model and requires rebuilding the computation.

This paper treats the pipeline's own structure as editable history. RetroC compiles a small, pure dataflow language into a versioned dataflow graph, the *Retroactive IR*, executes it once while recording per-record memo tables, and then accepts edit scripts that modify past input records, insert stages or delete stages. Each edit is type-checked and applied atomically, and its effect is propagated through the graph at the granularity of individual records.

A. Motivation

Two observations motivate the design. First, in a *pure* language a stage's output for a record is fully determined by that record's input value and the stage's function. Purity therefore makes record-level reuse sound without any runtime dependency tracing: if neither the input value nor the function changed, the cached output is still correct. Second, users who edit a pipeline need to *trust* and *understand* the result. An incremental engine that silently returns a different answer from a full rerun is worse than no engine at all, and one that cannot say what it recomputed is hard to debug. RetroC therefore pairs incremental propagation with a built-in verifier and a human-readable rebuild plan for every edit.

B. Problem Statement

Let P be a well-typed program, I its input data, and S the state left by executing P on I (outputs, memo tables and hashes). Given an edit e from the set {modify, insert or delete an input record; insert or delete a stage}, the problem is to produce outputs equal to those of a from-scratch run of the edited program, $Run(Apply(e, P), Apply(e, I))$, while doing work proportional to the records whose values actually change rather than to the size of the input times the number of stages. Invalid edits must be rejected with a diagnostic and must leave S unchanged.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

C. Contributions

This paper makes the following contributions:

- A pure dataflow language and a versioned intermediate representation in which input-record modification, insertion and deletion and stage insertion and deletion are first-class, type-checked, atomic compiler operations with an exact undo on failure (Section IV).
- A record-level propagation algorithm that combines per-record memoization, an order-independent collection hash updated in constant time per changed record, incrementally maintained integer sums and hash joins, and forwarding of only the records whose output changed, so that work per stage is proportional to the number of changed records (Sections IV–VI).
- A per-stage cost model that chooses between patching and recomputing, so that no edit kind pays for bookkeeping it cannot use (Section IV).
- An end-to-end verifier that compares propagated results with a fresh run of the textually rewritten program, and a differential testing harness that exercised 10,000 random programs and edit scripts (Section VII).
- An evaluation over 35 workload–edit scenarios against a full rerun and against stage-level invalidation, using a reproducible pinned and interleaved timing protocol, in which RetroC is faster than a full rerun in every scenario (Sections VIII–XI).

II. Literature Review

RetroC draws on several lines of work. This section summarises each and states how RetroC relates to it; Table I compares the approaches along the properties that matter for retroactive editing.

A. Batch Reprocessing and Workflow Systems

Data-parallel batch engines recompute results from their inputs, and some can recover lost partitions from recorded lineage [1]. Workflow management systems and build tools such as Make decide *which tasks* to rerun from a dependency graph between tasks or files [2]. Their unit of reuse is the whole task: a one-record change to a task's input reruns that task and every task after it in full. RetroC's stage-level baseline (B1 in Section VIII) models this behaviour.

B. Build Systems and Early Cutoff

Mokhov et al. characterise build systems by their scheduling and rebuilding strategies and identify *early cutoff*—stopping when a rebuilt target is unchanged—as a key property [3]. Build tools with dynamic dependencies and query-based compilers apply the same idea to tasks and to compiler queries. RetroC applies early cutoff at two levels: per record, through memo hits, and per stage, through the collection hash.

C. Self-Adjusting and Demand-Driven Computation

Self-adjusting computation records a dynamic dependence graph during execution and re-executes only the affected parts after an input change [4]; selective memoization keys memo entries on the values a computation actually uses [8]. Adapton makes incremental computation demand-driven and composable [5]. These systems are general-purpose and trace dependencies at runtime. RetroC is narrower: it exploits the static structure of a pure collection pipeline, so the dependency structure is the compiled graph itself and no tracing is needed. In exchange, it adds structural edits of the program, which are not part of these models.

D. Incremental Dataflow and View Maintenance

Differential dataflow represents collections as accumulated differences and propagates input changes through a fixed dataflow graph [6], built on the timely dataflow model of Naiad [7]. Database incremental view maintenance derives update rules for materialised views from query definitions [9]. Stream processors handle unbounded, time-advancing inputs [10]. All of these excel at input changes; none treats inserting or deleting an operator in an already-running computation as an incremental update.

E. Incremental Programs and Compilers

The incremental lambda calculus derives programs that map input changes to output changes for higher-order languages [11]. Reactive notebooks re-execute the cells that depend on an edited cell, which is again stage-level reuse. RetroC is a compiler in the conventional sense—lexer, parser, type checker, IR, bytecode—whose IR is designed to be *edited* after execution.

F. Retroactivity, Provenance and Testing

The term *retroactive* comes from retroactive data structures, which support inserting or deleting operations in the past of a data structure's history [12]; RetroC borrows the idea, not the technique. Its *explain* command relates to data provenance, which tracks why and where an output value came from [13]. Its verification strategy follows differential testing, which compares two implementations of the same specification on generated inputs [14]; [15], and metamorphic testing, which checks relations between related executions [16].

TABLE I. Comparison of Approaches to Recomputation After a Change

Approach	Input edits	Stage edits	Reuse unit	Verifier
Batch rerun [1]	rerun	rerun	none	–
Workflow / build [2]; [3]	yes	rerun stage	task	–
Self-adjusting [4]; [5]	yes	no	trace node	–
Differential dataflow [6]	yes	rebuild	difference	–
View maintenance [9]	yes	rebuild	delta	–
RetroC (this work)	**yes**	**yes**	**record**	**built in**

III. Research Gap

The literature offers either reuse at the level of whole stages or tasks, which is simple and correct but recomputes every record of every downstream stage, or fine-grained incremental computation over a computation whose shape is fixed when it is built. We found no system in which all of the following hold together:

- edits to past input records *and* structural edits that insert or delete stages are first-class, type-checked operations on the compiled program;
- reuse after such edits is tracked per record rather than per stage, with work proportional to the records that change;

- every edit produces an explicit rebuild plan stating what was recomputed and what was reused; and
- the incremental result is checked against a from-scratch run of the edited program.

RetroC is designed to fill this gap for a small but realistic pure dataflow language, and the evaluation in Section IX measures how far record-level reuse helps for each kind of edit.

IV. Proposed Methodology

A. The RetroC Language

A RetroC program is a sequence of typed `input` declarations, `let` stages, optional single-expression `fn` helpers and `output` declarations. Types are `int`, `float`, `bool`, `string`, arrays `[T]` and records `{f: T, ...}`. Collection builtins are `map`, `filter`, `reduce` (left fold), `tree_reduce` (associative fold) and `join` (equi-join); scalar helpers include `length`, `to_float`, `min`, `max` and `clamp`. The language is pure: there is no mutation, no input or output inside lambdas, no randomness and no recursion, which guarantees termination and makes every stage a pure function of its inputs. Lambdas may appear only as arguments to builtins, and their parameter types are inferred from the builtin. A lambda may also read another stage by name, for example `x - avg`. Every user-facing error has a code (E101–E601), a line and a column.

Listing 1. A sensor pipeline and an edit script

```
input readings: [{id: int, temp_c: float,
                  valid: bool}] = [ ... ];
let ok    = filter(readings, r => r.valid);
let temp_f = map(ok, r => r.temp_c*1.8 + 32.0);
let sum_f = reduce(temp_f, 0.0, (a, t) => a + t);
let avg   = sum_f / to_float(length(temp_f));
output avg;

retro modify readings[1].temp_c = 22.0
retro insert into readings :
      {id: 9, temp_c: 21.5, valid: true}
retro delete readings[3]
retro insert before temp_f as cal :
      map(_, r => {id: r.id,
                   temp_c: r.temp_c + 0.5,
                   valid: r.valid})
retro delete cal
```

Listing 1 shows a pipeline and an edit script. A record is named by its input and its RecordId; an inserted record receives a fresh RecordId that is never reused, so later edits keep referring to the same record. The underscore in an inserted stage stands for the collection the new stage will read. Edits apply in order and each receives an edit identifier used by the history log and by provenance queries.

B. The Retroactive IR

Each top-level stage becomes one node of a directed acyclic graph. A node records a stable identifier, a kind (INPUT, MAP, FILTER, REDUCE, TREE_REDUCE, JOIN, SCALAR, OUTPUT), its input edges, its lambda or expression, a structural hash of that function (*fnHash*), a hash of its current output (*outHash*), a version counter and a memo table. A builtin nested inside another is split into a hidden stage so that every builtin application is one node. Stages that a lambda reads by name become additional inputs of its node, after the collections it iterates, so the graph records every dependency. The graph keeps a topological order, recomputed with Kahn's algorithm after every structural edit [17]. Table II lists the invariants that the implementation maintains and that the tests check.

TABLE II. Invariants of the Retroactive IR

Id	Invariant
I1	The graph is acyclic and the stored order is topological.
I2	Node identifiers are unique and never reused.
I3	A memo entry is reused only if its input hash and memo key (function hash mixed with the output hashes of the stages it reads by name) both match.
I4	A node's version increases if and only if its output hash changes.
I5	After propagation every node is clean and equals the pure function of its inputs.
I6	Every node type-checks against its inputs after every edit.
I7	Collections keep source order; RecordIds are stable through MAP and FILTER, so every collection is sorted by RecordId.
I8	Running with an empty memo gives the same outputs as running with a warm memo.

C. Edit Semantics

Table III summarises the five edits. Each edit first checks its preconditions, then rewrites the graph or the input, recomputes the topological order and re-type-checks the affected stages. If any step fails, the rewrite is undone exactly—only the rewired edges and the added or removed node are restored—and the edit is reported with error E601. Because nothing else is mutated before these checks, the undo does not need to snapshot results or memo tables.

TABLE III. The Five Retroactive Edits

Edit	Precondition	Rewrite	Initial work set
Modify record	input, record and field exist; literal has the field's type	replace the field; patch the input hash	the modified record
Insert record	literal has the input's element type	append with a fresh RecordId	the new record
Delete record	input and RecordId exist	remove the record	the removed record
Insert stage	target is not an input; new lambda and target still type-check	new node N between P and target	all records of N
Delete stage	MAP/FILTER whose removal type-checks; not read by name; not sole source of an output	rewire consumers to the stage's input	all records, memo-filtered

D. Record-Level Propagation

Propagation maintains a *work set* consisting of the dirty nodes, the nodes that must be rescanned in full, and, for each record-tracking node, the set of pending RecordIds. Nodes are visited in topological order. A MAP or FILTER stage with few pending records patches its output in place: for each pending record it locates the input and output positions by binary search on RecordId (invariant I7), looks up the memo with key (hash of the input value, memo key), runs the lambda only on a miss, and inserts, replaces or removes the output record. The stage reports exactly the records whose *output* changed, including records that appeared or disappeared, and only those are passed to its consumers. A node that reads a changed stage by name is rescanned in full, because every one of its records may depend on that stage.

Folds and joins are maintained incrementally as well. A REDUCE whose combiner adds an integer term per element,

`(a, x) => a + t(x)`, stores each record's term and patches the sum by subtracting the old term and adding the new one, with 64-bit wrap-around arithmetic that is exactly associative. A JOIN is a hash join that keeps a per-record key index for each side and the layout of its output. When an edit keeps every join key and adds or removes no record, only the output records built from the changed inputs are rebuilt, in place; otherwise the join is rebuilt from the index, re-evaluating only the keys of changed records. Other REDUCE and SCALAR nodes reuse their memo when the hashes of all their inputs are unchanged, and `tree_reduce` updates one leaf-to-root path.

E. Cost Model

Bookkeeping is not free: a memo lookup or store costs about as much as evaluating a short lambda. RetroC therefore decides per stage, from quantities it already knows, whether patching pays off:

- **Patch or scan.** A MAP or FILTER stage patches when at most one eighth of its input is pending; otherwise one sequential pass with memo checks is cheaper than many binary searches.
- **Additive folds.** With k changed records out of n and a term of cost c, the sum is patched when $k(c + 2) < n\,c$, counting two lookups per patched record; otherwise it is re-folded.
- **Adaptive memo.** A full pass samples its first 64 memo lookups. If at least seven in eight miss, the rest would miss too, so lookups, stores and change tracking are switched off for that pass and its consumers rescan instead of patching.
- **New stages.** An inserted stage has nothing to look up; it stores no memo and does not list its records as changed. Its consumers rescan, still reusing every record the new stage left unchanged.

Every choice only selects between two correct ways of computing the same output, so the cost model cannot affect results, only speed. Memo entries left untouched by a disabled pass remain correct because they are keyed by input hash (I3).

F. Order-Independent Collection Hash

Early cutoff needs a hash of each stage's output. Recomputing an order-dependent hash after every edit costs time proportional to the collection. RetroC instead uses a commutative sum of per-record terms. For a record with identifier r and value hash $h(v)$,

$$t(r, v) = mix(mix(r) \oplus h(v)), \tag{1}$$

where *mix* is the SplitMix64 finaliser [18] and $\oplus$ is bitwise exclusive or. The hash of a collection C combines its size with the sum of its terms modulo 2^{64}:

$$H(C) = F(|C|, \Sigma_{(r,v)\in C}\, t(r, v) \bmod 2^{64}), \tag{2}$$

where F is an FNV-style combination. Replacing the value of record r updates the stored sum S in constant time,

$$S' = S - t(r, v_{\text{old}}) + t(r, v_{\text{new}}), \tag{3}$$

and insertion or removal adds or subtracts one term. Order is still captured: by invariant I7 every collection is sorted by unique RecordIds, so the set of (r, v) pairs determines the sequence. Sum-based multiset hashes are a known construction [19]. They are not collision-resistant against an adversary, but RetroC's inputs are not adversarial. If the terms behave like independent uniform 64-bit values, the probability that any of q compared states collide is at most $q^2/2^{65}$ by the birthday bound, which is below 10^{-8} even for $q = 10^5$ comparisons. The verifier compares full values, not hashes, so a collision cannot go undetected in a verified run.

G. Correctness Argument

Claim. For every well-typed program P, input I and valid edit e, propagating e from the state of executing P on I yields the same outputs as running the edited program on the edited input from an empty memo.

Proof sketch. By induction over the topological order. The work set initially contains every record whose input value or stage function changed, and every node that reads a changed stage by name. When a stage is visited, a memo entry is reused only if both its input hash and its memo key match (I3); since stages are pure and the memo key covers every stage the lambda reads, equal input under an equal key yields an equal output. A patched sum equals the re-folded sum because integer addition modulo 2^{64} is associative and commutative; a patched join keeps every key and therefore every matched pair. Records whose output is unchanged are not forwarded, and nodes not reached by the work set have unchanged inputs and functions, so their outputs are unchanged. The only gap is a hash collision, bounded above and closed by the verifier's full-value comparison.

H. Incremental Tree Reduction

For associative combiners that are not simple sums, `tree_reduce` stores a complete binary tree of partial results whose leaves are the records. Changing record k recomputes only the $\lceil \log_2 n \rceil$ internal nodes on the path from its leaf to the root: three combiner steps for $n = 8$ and ten for $n = 1{,}024$, both checked by unit tests.

V. System Architecture

RetroC is organised as a strict layered pipeline (Fig. 1). A module may include only modules to its left in the chain common → lexer → parser → sema → ir → bytecode → runtime → edit → reuse → verify → cli, so there are no dependency cycles. The front end turns source text into a typed program, the IR builder produces the Retroactive IR, lambda bodies are compiled to stack bytecode, and the graph executor performs the first run while filling memo tables. Edit scripts are parsed separately and applied by the edit engine; the propagator updates the graph, applies the cost model and produces the rebuild plan; the verifier optionally re-runs the rewritten program. Table IV lists the modules with their size and responsibility.

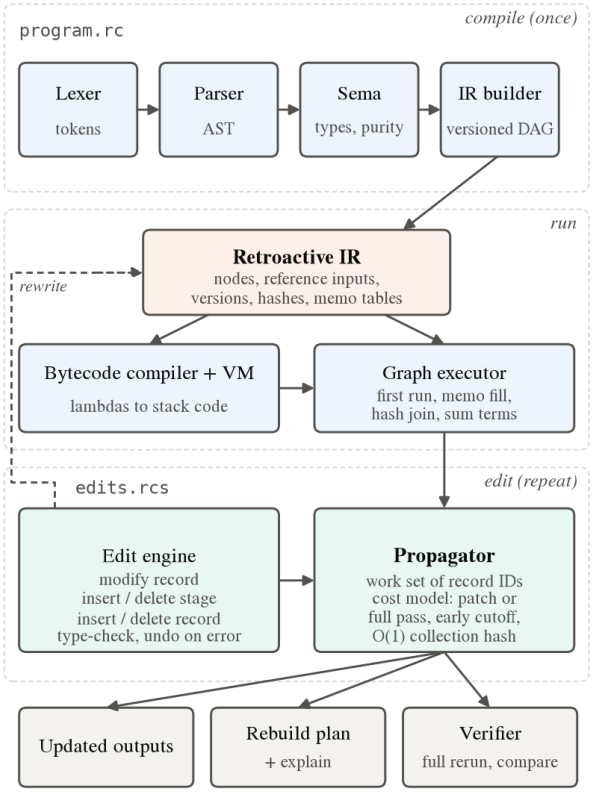


Fig. 1. Architecture of RetroC. The compile lane builds the Retroactive IR and bytecode; the execute lane performs the first run and fills the memo tables; the edit lane applies each retroactive edit as a graph or input rewrite, propagates it record by record under the cost model, and feeds the outputs, the rebuild plan and the verifier.

TABLE IV. MODULES OF THE IMPLEMENTATION

Module	LOC	Responsibility
common	412	values, structural hashing, collection hash, errors
lexer	397	tokens; errors E101–E102
parser	1,208	recursive descent, lambda lookahead; E201
sema	901	types, scopes, purity; E301–E403
ir	1,808	graph, nodes, memo, reference inputs, topological sort
bytecode	835	expression compiler and stack VM
runtime	1,028	first run, hash join, incremental sums, CSV loader
edit	987	edit-script parser, source rewriting
reuse	1,604	propagation, cost model, record edits, rebuild plan, B1 mode
verify	330	full-rerun comparison
cli	501	command-line driver

VI. ALGORITHM

Fig. 2 shows the complete control flow, from source text to verified outputs. Part A runs once. The program is lexed and parsed (step 1) and checked for types, scopes and purity (step 2); any error stops compilation with a code and a line:col position. The IR builder then produces the Retroactive IR in topological order (step 3), lambda bodies are compiled to bytecode (step 4), and the executor runs every stage once, filling the per-record memo tables and collection hashes (step 5).

Part B repeats for each edit in the script. The edit is read (step 6) and applied to a snapshot of the graph; stage inserts and deletes re-sort the stages with Kahn's algorithm [17], and the result is type-checked (step 7). An invalid edit, for example one that creates a cycle, restores the snapshot and reports E601, so the graph is unchanged. A valid edit seeds the work set W with the first affected stages and the pending record IDs; stages that read a changed stage by name are marked for a full scan (step 8). Stages are then visited in topological order (step 9). The cost model decides whether patching is cheaper; if not, one full pass is made, with the memo switched off when its lookups miss. Otherwise only the pending records are patched, and an integer sum swaps one term per record (step 10). The collection hash is updated in $O(1)$ per changed record (step 11). If the stage's output hash is unchanged, propagation stops there (early cutoff); otherwise its version is increased and only the changed record IDs are forwarded to its consumers (step 12). When no dirty stages remain, the changed outputs are refreshed and the rebuild plan is printed (step 13). After the last edit, `retroc verify` can rewrite the source, rerun it from scratch and compare the outputs (step 14).

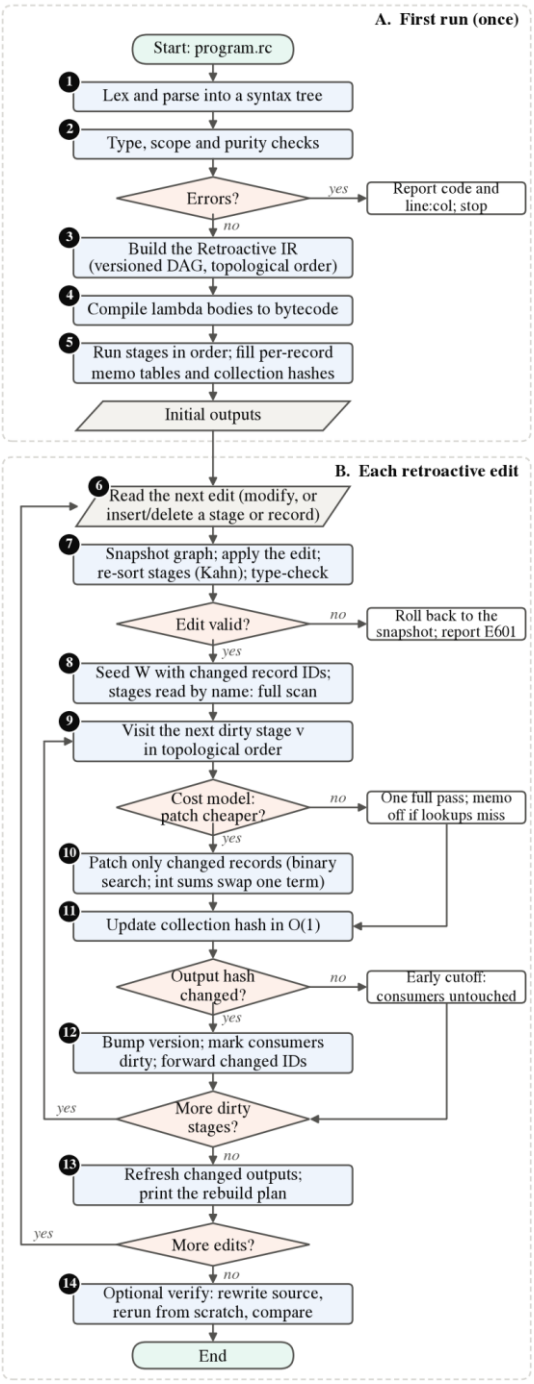


Fig. 2. End-to-end flowchart of RetroC. Part A compiles and runs the program once; Part B applies each retroactive edit and propagates it stage by stage, choosing between patching and a full pass with the cost model.

Algorithm 1 gives the shared propagation loop used by all five edits (steps 8–12 of Fig. 2), and Algorithm 2 the patch step for a MAP or FILTER stage.

Algorithm 1. PROPAGATE(G, W)

```
Input: graph G, work set W = (dirty, fullScan,
       pending: node -> set of RecordIds)
for v in topological order of G:
  if v is INPUT or v not in W.dirty: continue
  h0 <- v.outHash
  if v is MAP or FILTER:
     changed <- PATCH(v, W)              (Alg. 2)
  else if v is additive int REDUCE and
          k(c+2) < n c:  swap k terms in sum
  else if v is JOIN and no key changed:
     rebuild only the affected output pairs
  else if v is TREE_REDUCE:
     update leaves in W.pending[v]; ascend
  else recompute v unless its memo matches
     the hashes of all its inputs
  if hash(v) != h0 or v is new:         (I4)
     v.version <- v.version + 1
     for c in consumers(v):
        W.dirty <- W.dirty + {c}
        if c reads v by name or changed is
           unknown: W.fullScan += {c}
        else W.pending[c] += changed
  (otherwise: early cutoff, consumers untouched)
refresh only the outputs whose hash changed
```

Algorithm 2. PATCH(v, W) for MAP / FILTER

```
K <- W.pending[v];  in <- input of v
if v in W.fullScan or |K| > |in|/8:
   one pass over in; memo check per record;
   if 7/8 of the first 64 lookups miss:
      memo and tracking off; changed unknown
   else compare with old output -> changed
else for r in K:
   x <- binarySearch(in, r)
   y <- binarySearch(out, r)
   if x exists and memo[r] matches
        (hash(x), memoKey): reuse        (I3)
   else if x exists: run lambda; store memo
   insert / replace / erase y; add or
   subtract t(r, v) in the hash sum   (3)
   if the output record changed: add r
return changed
```

Complexity. For k pending records in a stage of n records, the patch path performs k binary searches and at most k lambda evaluations, $O(k \log n)$ plus the cost of inserting or erasing vector elements when a filter admits or drops a record. The hash update is $O(1)$ per changed record, an additive integer fold is patched with k binary searches into its sorted term list, and `tree_reduce` costs $O(k \log n)$. A join that keeps its keys touches only the output pairs of the k changed records. A single-record edit is therefore independent of n along the whole pipeline whenever each fold is additive or associative.

VII. Implementation Details

A. Technology Stack

RetroC is written in C++17 (about 10,000 lines in `src/`, plus about 3,800 lines of tests and benchmark code) and built with CMake and Ninja using GCC 16.2.0. Warnings are treated as errors. The only third-party component is the single-header doctest framework; the CSV loader is written in-house. Golden tests are driven by small Python scripts.

B. Runtime Representation

Values are immutable tagged unions with structural equality, a 64-bit structural hash and deterministic floating-point printing, so identical inputs produce byte-identical output. Lambda bodies run on a stack virtual machine whose operand stack keeps 64 slots inline and spills to the heap only for very large literals, so ordinary expressions never allocate. Each stage result stores its collection, sorted by RecordId, together with the running sum behind its collection hash. An additive fold keeps its per-record terms in a vector sorted by RecordId, and a join keeps, per side, a map from RecordId to its key, plus the (left, right) pair at each output position. A

static analysis of the compiled bytecode records which stages are read by name inside expressions, so a collection is materialised for expressions only when one reads it, and only its size when it is read through `length`.

C. Command-Line Interface

The `retroc` driver exposes `dump-tokens`, `dump-ast`, `check`, `dump-ir`, `run`, `edit`, `verify` and `explain`. The `edit` command prints a rebuild plan per edit listing, for every visited stage, how many records were evaluated and reused and which path the cost model chose; `explain` lists the edits that changed a given output and the first stage each change entered. Fig. 7–9 in Section IX show real sessions.

D. Verification Infrastructure

Correctness is checked at four levels: doctest unit tests per module; golden tests that compare the exact output of every compiler stage, of edits and of `explain`; the `verify` command, which rewrites the source text with the edits, compiles and runs it from scratch and compares every output by full structural equality; and a differential tester [14]; [15]. The differential tester generates well-typed programs of seven shapes (filter and map chains, a scalar reading a collection through `length`, branching graphs with `tree_reduce`, joins, integer folds, and lambdas that read other stages by name) over 5 to 200 records, applies one to four random edits (45% modify, 15% insert stage, 15% delete stage, 13% insert record, 12% delete record), and also injects invalid edits that must be rejected without changing any state. A metamorphic test checks that inserting an identity map leaves outputs unchanged [16].

VIII. Experimental Setup

A. Dataset

The benchmarks use four synthetic, seeded workloads produced by a generator included with the artifact (Table V). W1 models a sensor log: temperatures are uniform in [−20, 120], humidity in [0, 100], and a reading is valid if its temperature lies in [−10, 100]; it is generated at 10 k, 100 k and 1 M records to study scaling. W2 joins 500 users with 2,000 orders whose amounts are uniform in [10, 500]. W3 is a deep pipeline of twelve alternating map and filter stages over 100 k records with values uniform in [1, 100]. W4 is reduce-heavy: one map and one fold over 1 M records with values uniform in [1, 1000]. Fixed seeds make every run reproducible.

TABLE V. Workloads

Id	Records	Pipeline
W1	10k / 100k / 1M	filter → map → map → filter → reduce
W2	500 + 2,000	join → map → filter → reduce
W3	100k	12 alternating map/filter → reduce
W4	1M	map → reduce

B. Edit Scenarios

Six edit scenarios are applied to each workload: E-A modifies one input record; E-B modifies 1% of the records, capped at 100 sequential edits; E-C inserts a map near the start of the pipeline; E-D inserts a map near the end; E-E deletes a middle stage; and E-F inserts an identity map. W4 has no E-E scenario because deleting its only middle stage would change the element type. This yields 35 scenarios.

C. Baselines

Two baselines run in the same runtime, so that only the recomputation strategy differs. **B0** reruns the edited program from scratch with an empty memo. **B1** is stage-level invalidation: any change to a stage's input recomputes every record of that stage without per-record memo, as a workflow or build system would, while keeping whole-stage early cutoff. **B2** is RetroC. A comparison with an external incremental engine is left to future work, because it would measure a different language and runtime.

D. Hardware and Protocol

All measurements were taken on an Intel Core i7-13620H laptop processor (6 performance and 4 efficiency cores, 16 threads) with 15.6 GB of RAM under Windows 11, using a release build with -O2. Each system receives three warm-up runs followed by eleven timed runs, and the median is reported; a system whose single run exceeds 10 s receives one warm-up and five timed runs [20]. Only propagation (B1, B2) or execution (B0) is timed; parsing, compilation, data loading and graph cloning are excluded. The process is pinned to one performance core at high priority, because the same work ran about twice as slowly on an efficiency core, and B0, B2 and B1 are timed round-robin so that slow drift such as thermal throttling affects all three alike. Every scenario is additionally run through the verifier.

E. Evaluation Metrics

- **Wall-clock time** per edit script, median of eleven runs (min and max are recorded).
- **Speedup over a full rerun**, B0/B2, and **over stage-level invalidation**, B1/B2.
- **Operations** recomputed and reused, where an operation is one lambda evaluation or one folded element, and **reuse** = reused / (recomputed + reused).
- **First-run overhead** of recording memo tables and history, and **memo size** relative to input size.
- **Correctness**: verifier result per scenario, unit and golden tests, and differential trials.

IX. Results

A. Single-Record Edits

Fig. 3 shows the speedup of RetroC over a full rerun when one input record is modified. On W1 it is 525×, 5,444× and 53,400× at 10 k, 100 k and 1 M records; on the 12-stage W3 it is 10,617×, on the reduce-heavy W4 27,348×, and on the join workload W2 185×. Fig. 4 explains why the speedup grows with input size: a full rerun and stage-level invalidation both grow linearly, from 30.3 ms to 5,259.9 ms, while a RetroC edit stays at about 0.10 ms at every size. On W1-1M it recomputes 5 operations and reuses 4,043,541: the changed record passes through each stage once and the final sum swaps one term.

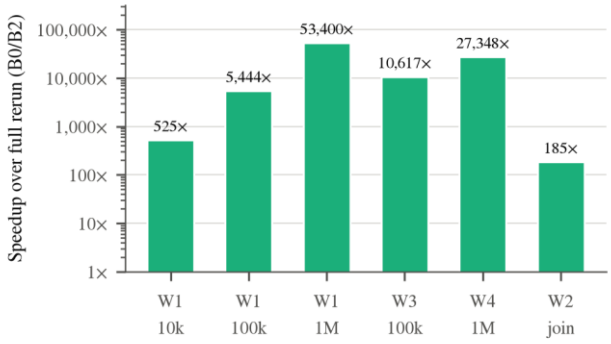


Fig. 3. Speedup of RetroC over a full rerun (B0/B2) for a single-record edit (E-A), logarithmic scale. Every workload is faster than a rerun, and the gain grows with input size.

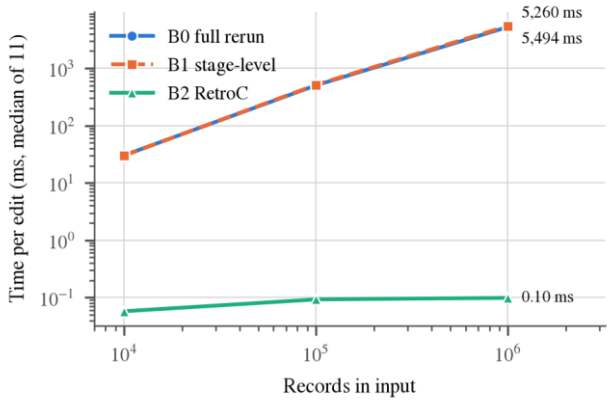


Fig. 4. Time per single-record edit on W1 versus input size (log–log). A full rerun (B0) and stage-level invalidation (B1) grow linearly; RetroC (B2) stays constant.

B. All Scenarios

Table VI lists all 35 scenarios and Fig. 5 shows both speedups as a matrix. RetroC is faster than a full rerun in every scenario, from 1.08× to 53,400×. Batches of sequential record edits (E-B) gain 6.23–1,065×, because each edit is patched on its own and costs a fraction of a millisecond. Inserts near the end of the pipeline gain 2.28–7.47×, identity inserts 4.03–7.85× and deletes 1.78–7.57×, because every stage before the edit point is reused. An insert at the very start (E-C) must evaluate the new stage on every record and changes every record downstream, so its gain of 1.08–1.43× comes from reusing the stages the change does not reach and from the cost model, which avoids per-record bookkeeping when nothing can be reused.

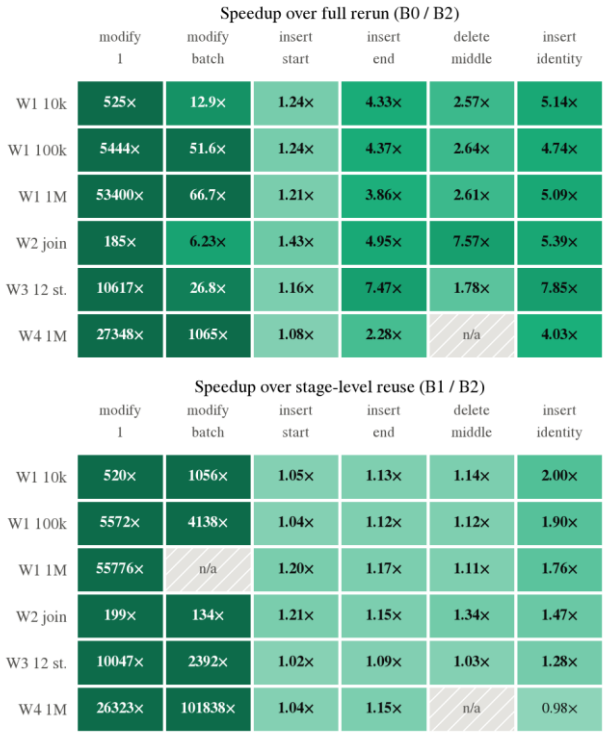


Fig. 5. Performance matrix: speedup of RetroC over a full rerun (top) and over stage-level invalidation (bottom) for every workload and edit scenario. Green cells are faster than the baseline; the colour scale is logarithmic and the median time of B2 is printed in each cell.

TABLE VI. MEDIAN TIME PER EDIT SCRIPT (MS) AND SPEEDUPS FOR ALL SCENARIOS

Work-load	Edit	B0	B1	B2	B0/B2	B1/B2
W1-10k	E-A	30.3	29.9	0.06	525	520
W1-10k	E-B	47.2	3,852.5	3.65	12.9	1,056
W1-10k	E-C	59.6	50.7	48.2	1.24	1.05
W1-10k	E-D	60.4	15.8	14.0	4.33	1.13
W1-10k	E-E	34.6	15.4	13.5	2.57	1.14
W1-10k	E-F	52.1	20.3	10.1	5.14	2.00
W1-100k	E-A	504.1	515.9	0.09	5,444	5,572
W1-100k	E-B	508.2	40,723.1*	9.84	51.6	4,138
W1-100k	E-C	624.9	521.0	503.4	1.24	1.04
W1-100k	E-D	611.4	156.9	139.8	4.37	1.12
W1-100k	E-E	399.4	168.9	151.0	2.64	1.12
W1-100k	E-F	608.3	244.4	128.4	4.74	1.90
W1-1M	E-A	5,259.9	5,493.9	0.10	53,400	55,776
W1-1M	E-B	5,385.3	–	80.8	66.7	–
W1-1M	E-C	8,300.3	8,236.5	6,842.9	1.21	1.20
W1-1M	E-D	8,157.8	2,468.3	2,113.8	3.86	1.17
W1-1M	E-E	3,958.0	1,688.1	1,516.7	2.61	1.11
W1-1M	E-F	8,168.3	2,822.6	1,603.7	5.09	1.76
W2	E-A	8.47	9.15	0.05	185	199
W2	E-B	8.56	184.8	1.38	6.23	134
W2	E-C	9.25	7.81	6.47	1.43	1.21
W2	E-D	9.23	2.15	1.86	4.95	1.15
W2	E-E	7.47	1.32	0.99	7.57	1.34
W2	E-F	10.0	2.74	1.86	5.39	1.47
W3	E-A	1,592.6	1,507.0	0.15	10,617	10,047
W3	E-B	1,728.7	154,281.0*	64.5	26.8	2,392
W3	E-C	1,988.7	1,743.2	1,712.3	1.16	1.02
W3	E-D	1,972.3	286.9	264.1	7.47	1.09
W3	E-E	1,566.7	911.5	881.5	1.78	1.03
W3	E-F	1,908.6	312.4	243.2	7.85	1.28
W4	E-A	1,392.0	1,339.8	0.05	27,348	26,323
W4	E-B	1,398.0	133,673.0*	1.31	1,065	101,838
W4	E-C	3,662.2	3,531.1	3,400.8	1.08	1.04
W4	E-D	2,049.4	1,029.6	899.2	2.28	1.15
W4	E-F	1,944.0	471.3	482.0	4.03	0.98

* single run above 10 s: one warm-up and median of five runs. – not run (estimated over one hour). All 35 scenarios passed the verifier.

C. Record-Level Versus Stage-Level Reuse

The bottom half of Fig. 5 isolates the value of record granularity by comparing RetroC with stage-level invalidation. For edits to input records the difference is large: 199–55,776× for a single edit and 134–101,838× for batches, because B1 recomputes every record of every downstream stage for each edit. For structural edits RetroC is ahead of or level with stage-level invalidation (0.98–2.00×): an inserted stage must see every record under either strategy, and the cost model ensures that RetroC adds no bookkeeping in that case, while identity inserts let it reuse the downstream stages record by record. The one scenario below 1.00×, W4 E-F (0.98×), is an identity map inserted directly before the final sum: both strategies run the new stage once over every record and both skip the sum because its input is unchanged, so they do identical work and the remaining difference is measurement noise (a separate pinned re-run of the same scenario measured 505.0 ms for stage-level invalidation and 506.2 ms for RetroC).

D. Effect of the Cost Model and Incremental Operators

Table VII compares the propagator before and after the cost model, the incremental sum and the hash join were added, using the same workloads, machine and protocol. The largest gains are on the scenarios that previously had to re-fold the whole input or re-run a nested-loop join: a single-record edit on W1-1M went from 10.0× to 53,400× faster than a rerun, a batch of 100 edits on W1-100k from 0.13× to 51.6×, and a single edit on the join workload from 0.40× to 185×. Structural edits that change every record moved from below parity to above it.

TABLE VII. SPEEDUP OVER A FULL RERUN BEFORE AND AFTER THE COST MODEL

Workload	Edit	Before	After
W1-100k	E-A	9.87×	5,444×
W1-1M	E-A	10.0×	53,400×
W1-100k	E-B	0.13×	51.6×
W3	E-B	0.23×	26.8×
W4	E-B	0.03×	1,065×
W2	E-A	0.40×	185×
W2	E-B	0.02×	6.23×
W1-100k	E-C	1.00×	1.24×
W3	E-C	0.77×	1.16×
W4	E-F	3.29×	4.03×

Before: previous propagator (nested-loop join, linear fold, no cost model). Same machine and protocol; B0/B2 medians.

An earlier step had already made propagation proportional to the changed records. Fig. 6 compares that design with the original one, which visited, hashed and copied every record of every dirty stage, on W1-100k under the earlier and less strict protocol (one warm-up and median of three, unpinned). The single-record speedup rose from 1.28× to 5.42×, the late insert from 1.31× to 5.21×, and the delete from 0.86× to 2.20×, while operation counts were unchanged: the work avoided was bookkeeping, not computation. Together, the two steps account for the results above.

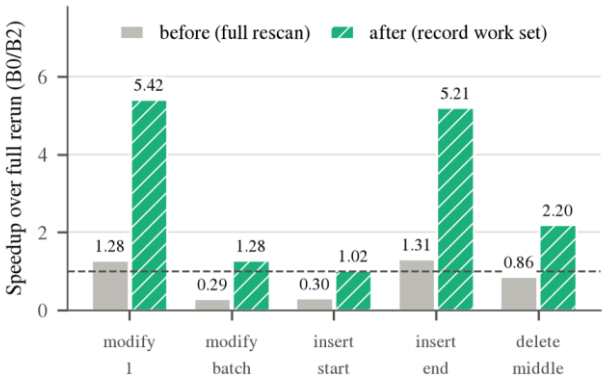


Fig. 6. Ablation on W1-100k (earlier protocol): speedup over a full rerun before and after record-proportional propagation, the first of the two optimisation steps.

E. Overheads

Recording memo tables and history during the first run of W1-100k raises its time from 479.2 ms to 569.7 ms, an overhead of 18.9%. The memo tables occupy 39.29 MB for 1.73 MB of CSV input, about 23× the input size, because each record's memo entry stores two hashes and a copy of the output value. The investment is recovered by the first edit: a single-record edit on W1-100k saves about 504 ms compared with a rerun, more than the 90 ms spent recording.

F. Correctness Results

TABLE VIII. CORRECTNESS EVIDENCE

Check	Result
Unit tests (doctest)	65 cases, 1,097 assertions pass
Test suites (CTest)	8 / 8 pass, golden outputs unchanged
Differential trials, 5 seeds	10,000 / 10,000 match rerun
Invalid-edit injection	all rejected, state unchanged
Planted bug (mutation)	detected at trial 16
Verifier on benchmarks	35 / 35 scenarios pass

Table VIII summarises the correctness evidence. To check that the differential tester can detect real propagation errors, we planted a defect—a record removed from a stage's output was not forwarded to its consumers—and the tester reported a mismatch at trial 16. The tester also found three defects during development of the new operators (an inserted stage wired to an output of the same name, a type check that ignored stages read by name, and a join patch applied to a record that had re-entered its input); each was fixed at its root cause and its failing case kept as a regression test.

G. The Prototype in Use

Fig. 7–9 show unedited terminal sessions of the `retroc` command-line tool on the demonstration programs; every line is program output. Fig. 7 runs a sensor pipeline and applies four retroactive edits, each followed by its rebuild plan. Fig. 8 verifies the edited result against a fresh run of the rewritten source and asks which edits changed an output. Fig. 9 inserts and deletes input records and shows `tree_reduce` updating one leaf-to-root path.

```
retroc: first run and four retroactive edits

$ retroc run demo/pipeline.rc
avg_temp_f = 73.174999999999997

$ retroc edit demo/pipeline.rc demo/edits.rcs
Edit #1: retro modify sensor_readings[1].temp_c = 22.0
Plan:
  valid_readings PARTIAL  evaluated      1 / 5       reused 4
Recomputed: 1 ops   Reused: 4 ops   Full-rerun cost: 5 ops
Reuse: 80.0%   Updated outputs: (none)
Edit #2: retro modify sensor_readings[1].valid = true
Plan:
  valid_readings PARTIAL  evaluated      1 / 5       reused 4
  temp_f     PARTIAL  evaluated      1 / 5       reused 4  (early
      cutoff on unchanged inputs)
  sum_f      FULL     evaluated      5 / 5       (reduce is O(n) in
      v1)
  count      FULL     evaluated      1 / 1
  avg_temp_f FULL     evaluated      1 / 1
Recomputed: 9 ops   Reused: 8 ops   Full-rerun cost: 17 ops
Reuse: 47.1%   Updated outputs: avg_temp_f
Edit #3: retro insert before temp_f as calibrated : map(_, r => { id
     : r . id , temp_c : r . temp_c + 0.5 , valid : r . valid })
Plan:
  calibrated NEW      evaluated      5 / 5
  temp_f     FULL     evaluated      5 / 5
  sum_f      FULL     evaluated      5 / 5       (reduce is O(n) in
      v1)
  count      FULL     evaluated      1 / 1
  avg_temp_f FULL     evaluated      1 / 1
Recomputed: 17 ops   Reused: 0 ops   Full-rerun cost: 17 ops
Reuse: 0.0%   Updated outputs: avg_temp_f
Edit #4: retro delete calibrated
Plan:
  temp_f     FULL     evaluated      5 / 5
  sum_f      FULL     evaluated      5 / 5       (reduce is O(n) in
      v1)
  count      FULL     evaluated      1 / 1
  avg_temp_f FULL     evaluated      1 / 1
Recomputed: 12 ops   Reused: 0 ops   Full-rerun cost: 12 ops
Reuse: 0.0%   Updated outputs: avg_temp_f
avg_temp_f = 72.859999999999985
```

Fig. 7. Terminal session: first run of the sensor pipeline and four retroactive edits with their rebuild plans (records evaluated and reused per stage).

```
retroc: verification and provenance

$ retroc verify demo/pipeline.rc demo/edits.rcs
Verification OK: all outputs match full rerun exactly.
avg_temp_f = 72.859999999999985

$ retroc explain demo/pipeline.rc demo/edits.rcs avg_temp_f
Explain avg_temp_f:
  Edit #2: retro modify sensor_readings[1].valid = true
    first stage: sensor_readings
    recomputed:  9 ops, reused: 8 ops
  Edit #3: retro insert before temp_f as calibrated : map(_, r => {
      id : r . id , temp_c : r . temp_c + 0.5 , valid : r . valid })
    first stage: calibrated
    recomputed:  17 ops, reused: 0 ops
  Edit #4: retro delete calibrated
    first stage: temp_f
    recomputed:  12 ops, reused: 0 ops
```

Fig. 8. Terminal session: `retroc verify` compares the propagated outputs with a fresh run of the rewritten program; `retroc explain` lists the edits that changed an output.

```
retroc: record insert/delete and tree_reduce

$ retroc edit demo/pipeline.rc demo/record_edits.rcs
Edit #1: retro insert into sensor_readings : { id: 106, temp_c:
    23.9, valid: true }
Plan:
  valid_readings PARTIAL  evaluated      1 / 6       reused 5
      (early cutoff on unchanged inputs)
  temp_f     PARTIAL  evaluated      1 / 5       reused 4  (early
      cutoff on unchanged inputs)
  sum_f      FULL     evaluated      5 / 5       (reduce is O(n) in
      v1)
  count      FULL     evaluated      1 / 1
  avg_temp_f FULL     evaluated      1 / 1
Recomputed: 9 ops   Reused: 9 ops   Full-rerun cost: 18 ops
Reuse: 50.0%   Updated outputs: avg_temp_f
Edit #2: retro delete sensor_readings[2]
Plan:
  valid_readings PARTIAL  evaluated      0 / 5       reused 5
      (early cutoff on unchanged inputs)
  temp_f     PARTIAL  evaluated      0 / 4       reused 4  (early
      cutoff on unchanged inputs)
  sum_f      FULL     evaluated      4 / 4       (reduce is O(n) in
      v1)
  count      FULL     evaluated      1 / 1
  avg_temp_f FULL     evaluated      1 / 1
Recomputed: 6 ops   Reused: 9 ops   Full-rerun cost: 15 ops
Reuse: 60.0%   Updated outputs: avg_temp_f
avg_temp_f = 73.129999999999995

$ retroc verify demo/pipeline.rc demo/record_edits.rcs
Verification OK: all outputs match full rerun exactly.
avg_temp_f = 73.129999999999995

$ retroc edit demo/tree_demo.rc demo/tree_demo.rcs
Edit #1: retro modify numbers[3] = 90
Plan:
  total      PARTIAL  evaluated      3 / 8       reused 5
      (tree_reduce O(log n) update)
Recomputed: 3 ops   Reused: 5 ops   Full-rerun cost: 8 ops
Reuse: 62.5%   Updated outputs: total
total = 410
```

Fig. 9. Terminal session: inserting and deleting input records, and an incremental `tree_reduce` update.

X. Comparison with Existing and Baseline Methods

Against the two baselines, RetroC is faster than a full rerun in all 35 scenarios; its results beat stage-level invalidation in 33 of 34 scenarios and are on par with it in the remaining one (W4 E-F, within 2%), where both strategies perform exactly the same work. For edits to input records, the case incremental systems are built for, it improves on a full rerun by up to 53,400× and on stage-level invalidation by up to 101,838×, and record granularity is what delivers the gain. For structural edits it improves on a full rerun whenever part of the pipeline is untouched by the edit, and stays ahead of stage-level invalidation because the cost model removes bookkeeping that cannot pay off.

Qualitatively (Table I), RetroC is the only approach among those surveyed in which inserting or deleting a stage is an incremental, type-checked operation on an already-executed computation, and the only one with a built-in check against a full rerun. Differential dataflow and incremental view maintenance are mature, highly optimised engines for input changes [6]; [9]; we did not measure them, and they target distributed, high-throughput settings that RetroC does not. RetroC's contribution is the edit model and its verified, explainable record-level propagation.

XI. Discussion

Why the speedup grows with size. With the incremental sum and hash join, every stage on the path of a changed record does constant or logarithmic work, so the cost of an edit no longer depends on the input size while the cost of a rerun

grows linearly. This is the property incremental computation aims for, and Fig. 4 shows it directly.

Batches. Sequential edits are now patched one by one at a fraction of a millisecond each, so a batch of 100 edits is still 6.23–1,065× faster than one rerun. Stage-level invalidation must recompute the pipeline once per edit, which is why its batches are the slowest configuration in Table VI.

Structural edits. An inserted stage must see every record, so the achievable speedup is bounded by the share of the pipeline the edit leaves untouched: an insert at the very start leaves little, an insert near the end or an identity insert leaves most. The measured gains follow exactly this order, and the cost model keeps per-record bookkeeping out of the cases where it cannot pay off.

Measurement. Hybrid laptop processors produced up to threefold timing differences between otherwise identical runs, depending on whether the operating system scheduled the benchmark on a performance or an efficiency core and on thermal state. Pinning to one performance core and timing all systems round-robin reduced the spread within a scenario to about ±10%. We recommend both practices for incremental-computation benchmarks on consumer hardware.

Threats to validity. The workloads are synthetic, the language is small, and all measurements come from one machine. The results support claims about this design on these workloads; Section XII lists the remaining scope boundaries.

XII. Limitations

- Memo tables use about 23× the input size, and the first run is 19% slower; execution is single-threaded and in memory.
- Only integer sums are maintained incrementally; floating-point folds are re-folded, because floating-point addition is not associative, and other associative folds use `tree_reduce`.
- An insert at the very start of a pipeline changes every record, so its gain is modest by nature.
- The evaluation uses synthetic workloads on one machine and no external incremental engine.
- Edits cannot replace a stage in one step or change control flow.
- Memory-safety sanitizers could not be run with the MinGW toolchain used.

XIII. Conclusion

RetroC shows that modifying, inserting and deleting past input records and inserting and deleting stages can be handled as typed, atomic, first-class operations of a compiler for a pure dataflow language, and that their effects can be propagated record by record with work proportional to what changes. A single-record edit runs 525–53,400× faster than a full rerun on a sensor pipeline from 10 k to 1 M records, 10,617× faster on a 12-stage pipeline and 185× faster on a join, and every one of the 35 measured scenarios is faster than a full rerun and either faster than or on par with stage-level invalidation. Every result matched a full rerun across unit, golden and benchmark tests and 10,000 random edit scripts. A per-stage cost model, an incremental integer sum and an incremental hash join were the keys to making every kind of edit pay off.

XIV. Future Work

- **Real datasets:** evaluate on public data pipelines and longer, realistic edit histories.
- **External baseline:** compare input-edit performance with an established incremental dataflow engine.
- **Compact memo tables:** store hashes instead of output copies where outputs can be recomputed cheaply.
- **Richer edits:** stage replacement in one step and edits to helper functions.
- **Parallel propagation** across independent branches of the graph.

Acknowledgment

The authors thank the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, for the facilities and support provided for this work.

References

[1] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.

[2] J. Liu, E. Pacitti, P. Valduriez, and M. Mattoso, "A survey of data-intensive scientific workflow management," *J. Grid Comput.*, vol. 13, no. 4, pp. 457–493, 2015, doi: 10.1007/s10723-015-9329-8.

[3] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proc. ACM Program. Lang.*, vol. 2, no. ICFP, pp. 1–29, 2018, doi: 10.1145/3236774.

[4] U. A. Acar, G. E. Blelloch, and R. Harper, "Adaptive functional programming," in *Proc. 29th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL)*, 2002, pp. 247–259, doi: 10.1145/503272.503296.

[5] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, demand-driven incremental computation," in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2014, pp. 156–166, doi: 10.1145/2594291.2594324.

[6] M. Abadi, F. McSherry, and G. D. Plotkin, "Foundations of differential dataflow," in *Foundations of Software Science and Computation Structures (FoSSaCS)*, Lecture Notes in Computer Science, vol. 9034. Berlin, Germany: Springer, 2015, pp. 71–83, doi: 10.1007/978-3-662-46678-0_5.

[7] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi, "Naiad: A timely dataflow system," in *Proc. 24th ACM Symp. Oper. Syst. Princ. (SOSP)*, 2013, pp. 439–455, doi: 10.1145/2517349.2522738.

[8] U. A. Acar, G. E. Blelloch, and R. Harper, "Selective memoization," in *Proc. 30th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL)*, 2003, pp. 14–25, doi: 10.1145/604131.604133.

[9] J. A. Blakeley, P.-Å. Larson, and F. W. Tompa, "Efficiently updating materialized views," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1986, pp. 61–71, doi: 10.1145/16894.16861.

[10] A. Toshniwal *et al.*, "Storm@twitter," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 147–156, doi: 10.1145/2588555.2595641.

[11] Y. Cai, P. G. Giarrusso, T. Rendel, and K. Ostermann, "A theory of changes for higher-order languages: Incrementalizing λ-calculi by static differentiation," in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2014, pp. 145–155, doi: 10.1145/2594291.2594304.

[12] E. D. Demaine, J. Iacono, and S. Langerman, "Retroactive data structures," *ACM Trans. Algorithms*, vol. 3, no. 2, Art. no. 13, 2007, doi: 10.1145/1240233.1240236.

[13] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory—ICDT 2001*, Lecture Notes in Computer Science, vol. 1973. Berlin, Germany: Springer, 2001, pp. 316–330, doi: 10.1007/3-540-44503-X_20.

[14] J. Chen *et al.*, "A survey of compiler testing," *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–36, 2020, doi: 10.1145/3363562.

[15] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proc. 32nd ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2011, pp. 283–294, doi: 10.1145/1993498.1993532.

[16] T. Y. Chen *et al.*, “Metamorphic testing: A review of challenges and opportunities,” *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–27, 2018, doi: 10.1145/3143561.

[17] A. B. Kahn, “Topological sorting of large networks,” *Commun. ACM*, vol. 5, no. 11, pp. 558–562, 1962, doi: 10.1145/368996.369025.

[18] G. L. Steele, D. Lea, and C. H. Flood, “Fast splittable pseudorandom number generators,” in *Proc. ACM Int. Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*, 2014, pp. 453–472, doi: 10.1145/2660193.2660195.

[19] D. Clarke, S. Devadas, M. van Dijk, B. Gassend, and G. E. Suh, “Incremental multiset hash functions and their application to memory integrity checking,” in *Advances in Cryptology—ASIACRYPT 2003*, Lecture Notes in Computer Science, vol. 2894. Berlin, Germany: Springer, 2003, pp. 188–207, doi: 10.1007/978-3-540-40061-5_12.

[20] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*, 2007, pp. 57–76, doi: 10.1145/1297027.1297033.